ROBOTICS SUMMER SCHOOL - FUNDAMENTALS

Mathematics for Robotics

Probability and estimation

Krish Pandya

Morning session

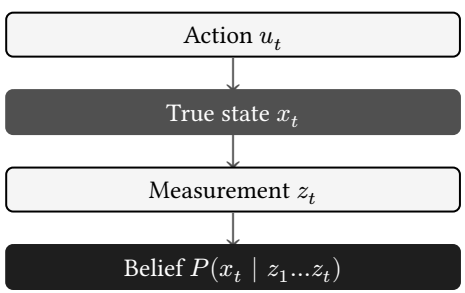
Determinism

The classical sense-think-act paradigm assumes the robot knows its location. It models the world with an exact equation $x_t = f(x_{t-1}, u_t)$.

Physical systems break this assumption. Sensors output noise. Actuators accumulate drift. A robot never knows its exact state. It possesses a probability distribution over the state space.

- State (x): The hidden ground truth of the system.
- Measurement (z): The noisy sensor output.
- Belief ($bel(x)$): The probability distribution over all possible states.

To tackle this idea, we model systems using Probability!



If sensors return noisy data, how does the robot ever make a reliable decision?

Probability: two interpretations

FREQUENTIST

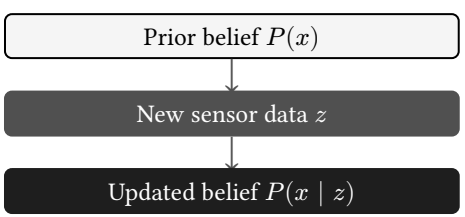
Probability is the long-run frequency of an event over repeated trials. If a fair coin is flipped 10,000 times, heads appears approximately 5,000 times. This reading requires a repeatable experiment.

BAYESIAN

Probability is a degree of belief. It quantifies what a rational agent knows about an event before and after observing data. No repeated experiment is required.

A robot's current position cannot be measured in a long-run frequency sense. The robot exists at one location. The Bayesian reading is the only coherent interpretation for state estimation.

The Bayesian agent starts with a prior belief. Every new sensor reading updates that belief through Bayes' rule. The distribution narrows as evidence accumulates.



This update cycle runs continuously as the robot moves through the world. Bayesian probability is not a static property of the system; it changes every time the robot gathers information.

State spaces and beliefs

DISCRETE SPACES

Binary states like a door being open or closed. The probability is a single number between 0 and 1.

$$P(x = \text{open}) = 0.8$$

CONTINUOUS SPACES

Variables like position (x, y, θ) exist in continuous space. The belief is a probability density function. Integrating the function over an area gives the probability that the robot is inside that specific area.

```python
# A belief over a 1D state space
import numpy as np

state_space = np.linspace(0, 10, 100)
belief = initialize_uniform(state_space)

# The sum of all probabilities is 1
assert np.isclose(np.sum(belief), 1.0)
```

The robot must track its belief across every possible state simultaneously, updating the entire distribution at every time step.

The Markov assumption

Without simplifying, the current belief depends on every state, control, and measurement ever recorded. The computation complexity would grow exponentially as the robot would operate.

The Markov assumption removes this dependency. The current state x_t depends **only** on the immediately preceding state x_{t-1} and the control input u_t, not on the full history.

$$P(x_t \mid x_{0:t-1}, u_{1:t}, z_{1:t-1}) = P(x_t \mid x_{t-1}, u_t)$$

A measurement z_t depends only on the current state x_t.

$$P(z_t \mid x_{0:t}, u_{1:t}, z_{1:t-1}) = P(z_t \mid x_t)$$

These two conditional independence statements make the Bayes filter possible.
Each update requires only the previous belief, the current control input, and the current measurement.

Violations cause filter divergence. A slipping wheel introduces time correlations that the motion model does not capture. The position estimate drifts.
This is why SLAM systems model wheel slip and IMU bias as additional state variables rather than ignoring them.

The Markov assumption says the future is independent of the past given the present. Name one situation where a mobile robot breaks this assumption.

Conditional probability

Bayes' rule updates the belief distribution when new sensor data arrives.

$$P(x \mid z) = \frac{P(z \mid x)P(x)}{P(z)}$$

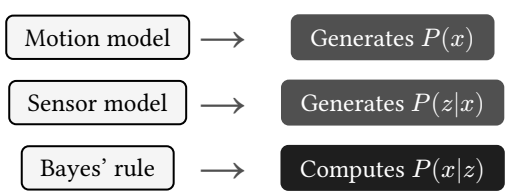
The denominator $P(z)$ is a constant η that ensures the final probabilities sum to 1.

THE GENERATIVE MODEL: $P(z \mid x)$

The sensor model maps states to expected measurements. It answers a specific question: If the robot is exactly at position x, what is the probability that the LiDAR scan looks like z?

THE PRIOR: $P(x)$

The motion model maps past states to current states. It provides an estimate of where the robot is located based entirely on odometry, evaluated before reading the new sensor data.



The Bayes filter

The Bayes filter implements Bayes' rule in a recursive algorithm. It runs continuously in a two-step loop.

1. Predict step: Push the state distribution forward using the physical motion equations and control inputs. This increases uncertainty.
2. Update step: Correct the state distribution using the measurement data and the generative sensor model. This decreases uncertainty.

The output of the update step becomes the input prior for the next predict step.

```python
def bayes_filter(bel_prev, u, z):
    # 1. Predict (Motion model)
    bel_bar = predict_state(bel_prev, u)

    # 2. Update (Sensor model)
    likelihood = compute_p_z_given_x(z, bel_bar)
    bel_new = eta * likelihood * bel_bar

    return bel_new
```

This recursive loop is the mathematical engine behind modern state estimation architectures.

If the prior predicts the wall is 2 meters away, but the sensor reads 10 meters, what should the robot believe?

Maximum likelihood estimation

MAXIMUM LIKELIHOOD ESTIMATION (MLE)

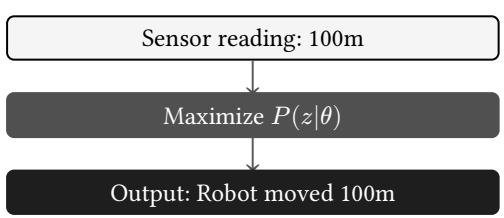
MLE finds the single state θ that makes the observed data most probable.

$$\hat{\theta}_{\text{MLE}} = \arg\max_{\theta} P(z \mid \theta)$$

This method relies entirely on the generative sensor model $P(z|\theta)$. It ignores the prior completely.

If a LiDAR ray hits a glass window and reports a 100-meter distance, the sensor model states the robot is 100 meters away from the wall. MLE accepts this data and assumes the robot teleported.

MLE operates with no memory and no physical constraints. It optimizes solely for the best fit to the current sensor reading.



Maximum a posteriori

MAXIMUM A POSTERIORI (MAP)

MAP estimation includes the prior $P(\theta)$ in the optimization objective.

$$\hat{\theta}_{\text{MAP}} = \arg\max_{\theta} P(z \mid \theta) P(\theta)$$

The prior anchors the estimate. It **regularizes** the sensor data against the known laws of physics.

If $P(z|\theta)$ suggests the robot teleported 100 meters in 0.1 seconds, the motion prior $P(\theta)$ assigns teleportation a probability of exactly zero.

The product of the two probabilities becomes zero, and the MAP estimator rejects the false sensor reading !

MAP estimation translates directly to machine learning 😺
It is mathematically equivalent to running MLE with L_2 (Ridge) (Gaussian Prior) or L_1 (Lasso) (Laplace Prior) regularization. The prior provides the weight decay penalty.

Why do SLAM papers model every spatial variable as a normal distribution?

The Gaussian distribution

State estimation papers and Kalman filters model noise using normal distributions, denoted as $\mathcal{N}(\mu, \Sigma)$.

The central limit theorem states that the sum of many independent random variables tends toward a normal distribution. Hardware noise stems from independent thermal and electrical fluctuations.

The mean μ tracks the expected state. The covariance matrix Σ tracks the uncertainty across multiple dimensions.

COMPUTATIONAL LIMITS

A robot operates at high frequencies, processing thousands of sensor data points per second.

Computing joint probabilities requires multiplying thousands of numbers that are smaller than 1. Processors experience arithmetic underflow and round the probability to zero.

The Gaussian distribution solves this hardware limitation.

Mean and variance

MEAN

The mean μ is the expected value. It is the robot's best single estimate of the state.

$$\mu = E[x]$$

VARIANCE

Variance measures the spread around the mean.

$$\sigma^2 = E[(x - \mu)^2]$$

Large variance indicates high uncertainty. Small variance indicates high confidence.

```python
# Processing 1D LiDAR distance measurements
import numpy as np

z = np.array([4.12, 4.15, 4.08, 4.11, 4.14])

# The estimate
mu = np.mean(z)

# The uncertainty
sigma_sq = np.var(z)
```

The mean indicates where the robot thinks it is. The variance indicates how wrong that estimate might be.

Multivariate states

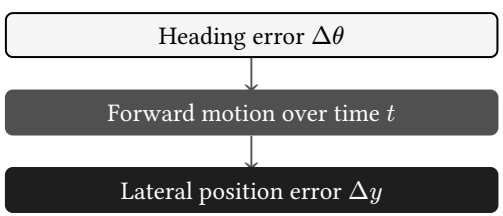
STATE VECTORS

A mobile robot tracks multiple variables simultaneously. The state is a vector x in $\mathbb{R}^n$.

$$x = (x \;\; y \;\; \theta \;\; v)^T$$

JOINT UNCERTAINTY

Tracking individual variances is insufficient. Variables are physically linked. An error in heading θ leads directly to an error in the lateral position y as the robot moves forward.



The estimator must track uncertainty across the entire vector space, mapping how an error in one dimension bleeds into another.

Covariance

COUPLED MOVEMENT

Covariance measures how two variables vary together.

$$\operatorname{cov}(x, y) = E\big[(x - \mu_x)(y - \mu_y)\big]$$

- Positive: The variables rise together.
- Negative: One rises while the other falls.
- Near zero: The variables exhibit independent linear motion.

If a drone pitches forward and its forward velocity increases, pitch and velocity have a positive covariance. The estimation algorithm uses this mathematical link to update velocity when pitch is measured.

The covariance matrix

MATRIX STRUCTURE

For a state vector x, uncertainty is organized into an $n \times n$ symmetric matrix Σ.

$$\Sigma = E\left[(x-\mu)(x-\mu)^T\right]$$

MATRIX COMPONENTS

The diagonal entries contain the individual variances σ_i^2. The off-diagonal entries contain the covariances $\mathrm{cov}\left(x_i, x_j\right)$ between different state variables.

DIAGONAL Σ: INDEPENDENT

The uncertainty ellipsoid aligns with the axes.

$$\Sigma = \begin{pmatrix} \sigma_x^2 & 0 \\ 0 & \sigma_y^2 \end{pmatrix}$$

FULL Σ: CORRELATED

The uncertainty ellipsoid rotates.

$$\Sigma = \begin{pmatrix} \sigma_x^2 & \mathrm{cov}(x,y) \\ \mathrm{cov}(x,y) & \sigma_y^2 \end{pmatrix}$$

Σ defines how uncertain each variable is and how that uncertainty couples with every other variable.

Raw data and the mean vector

THE BATCH DATASET

Assume a robot collects five independent measurements of its state $x = \begin{pmatrix} x & y & \theta & v \end{pmatrix}^T$.

$$\begin{pmatrix} 10.5 & 5.4 & 0.1 & 1.6 \\ 9.2 & 4.3 & -0.2 & 1.2 \\ 10.1 & 5.1 & 0.0 & 1.5 \\ 10.8 & 5.6 & 0.3 & 1.8 \\ 9.4 & 4.6 & -0.2 & 1.4 \end{pmatrix}$$

COMPUTING THE MEAN

The mean vector μ is the average of each column. It forms the best estimate of the current state.

$$\mu = \begin{pmatrix} 10.0 & 5.0 & 0.0 & 1.5 \end{pmatrix}^T$$

In reality, a Kalman filter does not store this raw array. It updates μ recursively as each new reading arrives. The array is shown here to illustrate the origin of the numbers.

Deriving the covariance matrix

THE MATRIX STRUCTURE

The covariance matrix Σ evaluates how those five raw data points vary against each other.

$$\Sigma = \begin{pmatrix} \sigma_x^2 & \text{cov}(x,y) & \text{cov}(x,\theta) & \text{cov}(x,v) \\ \text{cov}(y,x) & \sigma_y^2 & \text{cov}(y,\theta) & \text{cov}(y,v) \\ \text{cov}(\theta,x) & \text{cov}(\theta,y) & \sigma_\theta^2 & \text{cov}(\theta,v) \\ \text{cov}(v,x) & \text{cov}(v,y) & \text{cov}(v,\theta) & \sigma_v^2 \end{pmatrix}$$

THE COMPUTED RESULT

Processing the five data points yields a specific 4×4 geometry of uncertainty.

$$\Sigma = \begin{pmatrix} 0.48 & 0.37 & 0.14 & 0.15 \\ 0.37 & 0.30 & 0.11 & 0.12 \\ 0.14 & 0.11 & 0.05 & 0.05 \\ 0.15 & 0.12 & 0.05 & 0.05 \end{pmatrix}$$

Notice the positive values. In the raw data, when x was high (10.8), y was also high (5.6). The matrix captures this physical coupling directly.

Correlation

NORMALIZED COVARIANCE

Correlation is covariance divided by the product of the individual standard deviations.

$$\rho_{xy} = \frac{\operatorname{cov}(x, y)}{\sigma_x \sigma_y}$$

UNIT CANCELLATION

Covariance carries physical units. Multiplying a distance by an angle yields units like meters $\times$ radians. Correlation divides these units out.

THE CORRELATION MATRIX

Converting the full covariance matrix into a correlation matrix R bounds every element between -1 and 1. The diagonal is always exactly 1.

$$R = \begin{pmatrix} 1 & \rho_{12} & \cdots & \rho_{1n} \\ \rho_{21} & 1 & \cdots & \rho_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ \rho_{n1} & \rho_{n2} & \cdots & 1 \end{pmatrix}$$

Correlation provides a clean, comparable metric for linear dependence, completely independent of sensor scale.

The log-likelihood mechanism

THE GAUSSIAN PDF

The probability of a measurement z given a state x is modeled as a multivariate Gaussian distribution.

$$P(z|x) = \frac{1}{\sqrt{|2\pi\Sigma|}} \exp\left(-\frac{1}{2}(z - h(x))^T \Sigma^{-1} (z - h(x))\right)$$

LOGARITHMIC CONVERSION

To maximize this probability, estimators compute the negative natural logarithm. The logarithm is monotonic, so the location of the optimal state does not change.

$$-\ln(P(z|x)) = -\ln(\text{const}) + \frac{1}{2}(z - h(x))^T \Sigma^{-1} (z - h(x))$$

DROPPING CONSTANTS

The normalization constant $\frac{1}{\sqrt{|2\pi\Sigma|}}$ does not depend on the state x. When the algorithm optimizes for x, it ignores this constant.

Taking the negative log strips away the exponential function and the constant. It isolates the exponent.
This mathematical operation converts a probability maximization problem into an error minimization problem.

Probability to optimization

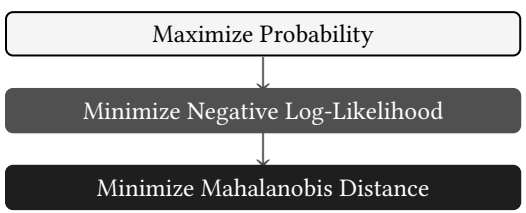
THE OBJECTIVE FUNCTION

The isolated exponent is the Mahalanobis distance. It is the core objective function $J(x)$ for non-linear least-squares optimization.

$$J(x) = (z - h(x))^T \Sigma^{-1} (z - h(x))$$

ALGORITHMIC CONTEXT

Finding the peak of a high-dimensional probability distribution is computationally expensive.
Finding the minimum of a quadratic bowl $J(x)$ is highly efficient.
SLAM backends compute the Jacobian of $h(x)$ and use solvers like Gauss-Newton or Levenberg-Marquardt to step toward the minimum.



The information matrix Σ^{-1} acts as a spatial weight. A noisy sensor produces a large variance, resulting in a small information weight. The optimizer allows large errors in that direction without penalty.

Reference materials

LECTURES

Cyrill Stachniss: Introduction to Mobile Robotics. Lecture 2 covers the mathematical derivations of the Bayes Filter. Watch the lecture.

LITERATURE

Thrun, Burgard, Fox: Probabilistic Robotics. Chapters 1 through 3 define the foundational equations for state estimation in robotics.

Emma Benjaminson: MLE vs MAP. This article provides numerical examples using a simulated wall-following robot. Read the article.

Roger Labbe: Kalman and Bayesian Filters in Python. Free interactive Jupyter notebooks with complete implementations of the discrete Bayes filter, Kalman filter, and particle filter. Read the book.